

CZ2006/CE2006 Software Engineering

Lecture 13: Object Design

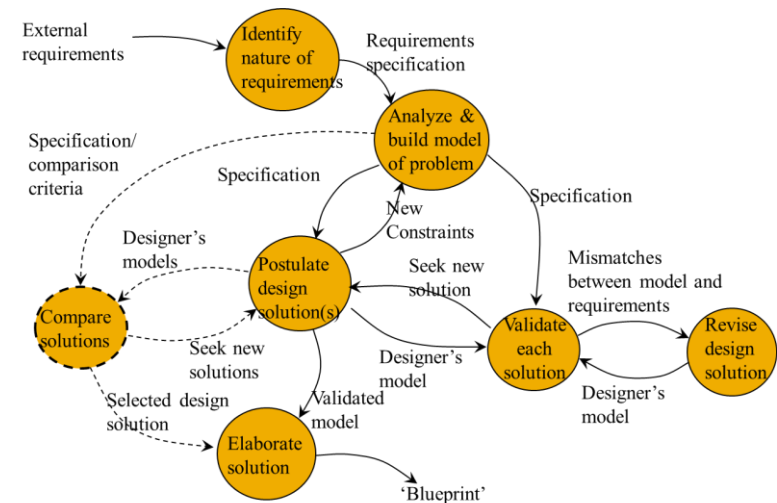
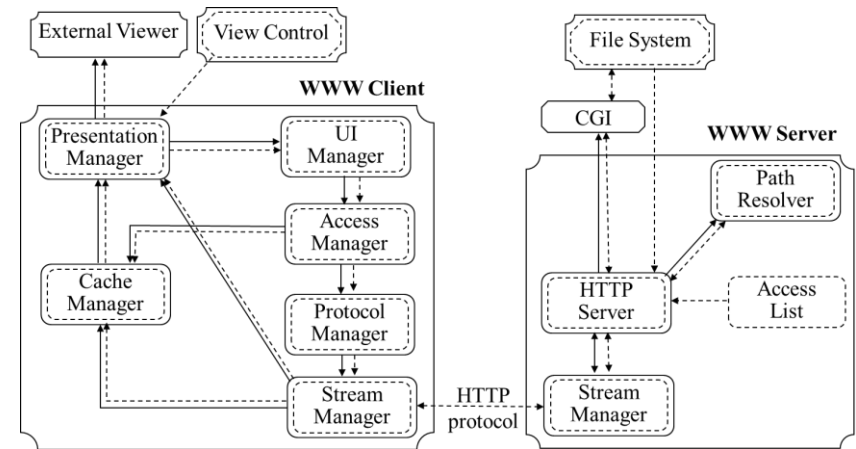
Liu Yang

Recap of the System Design

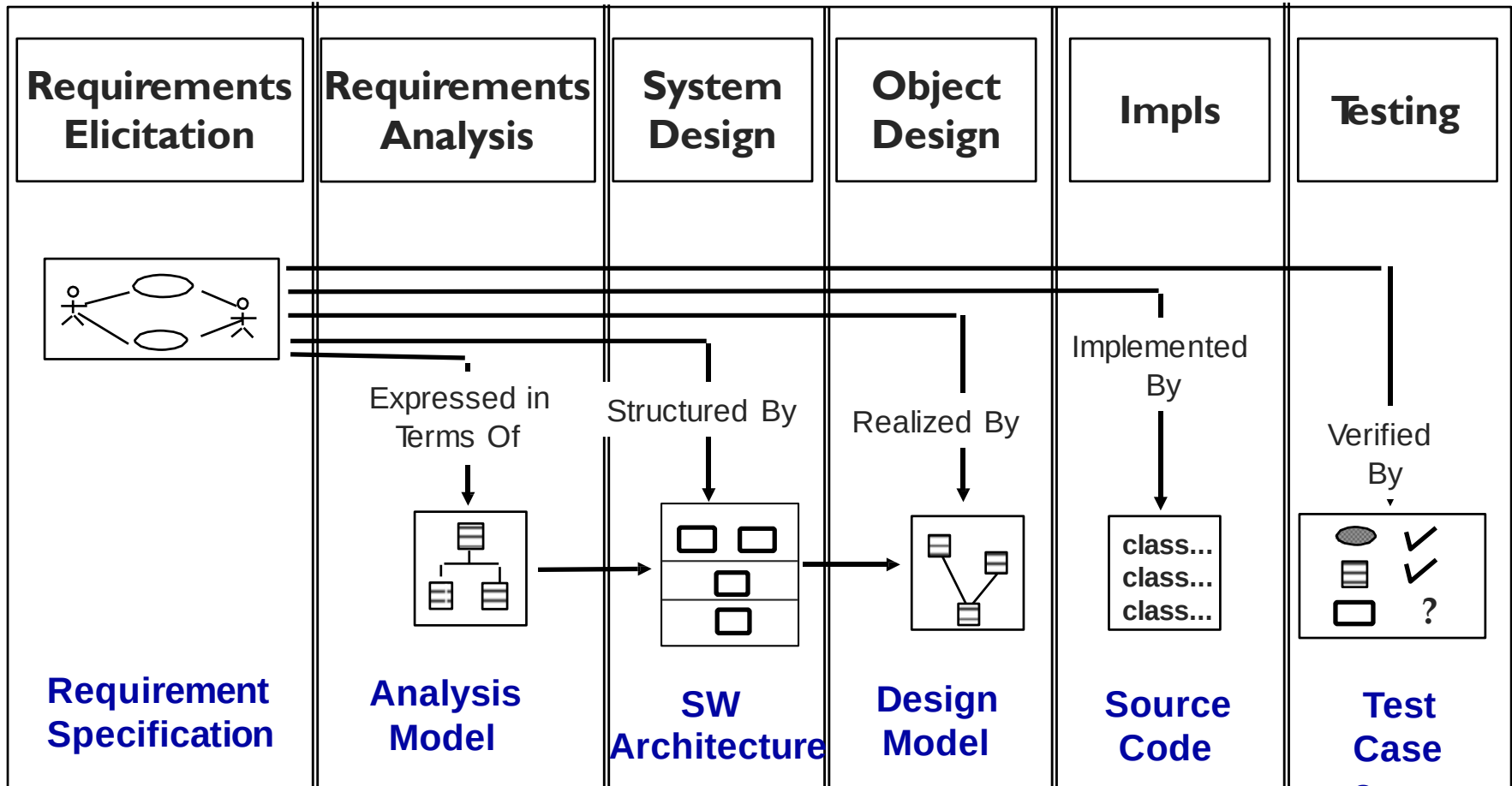
Why

What

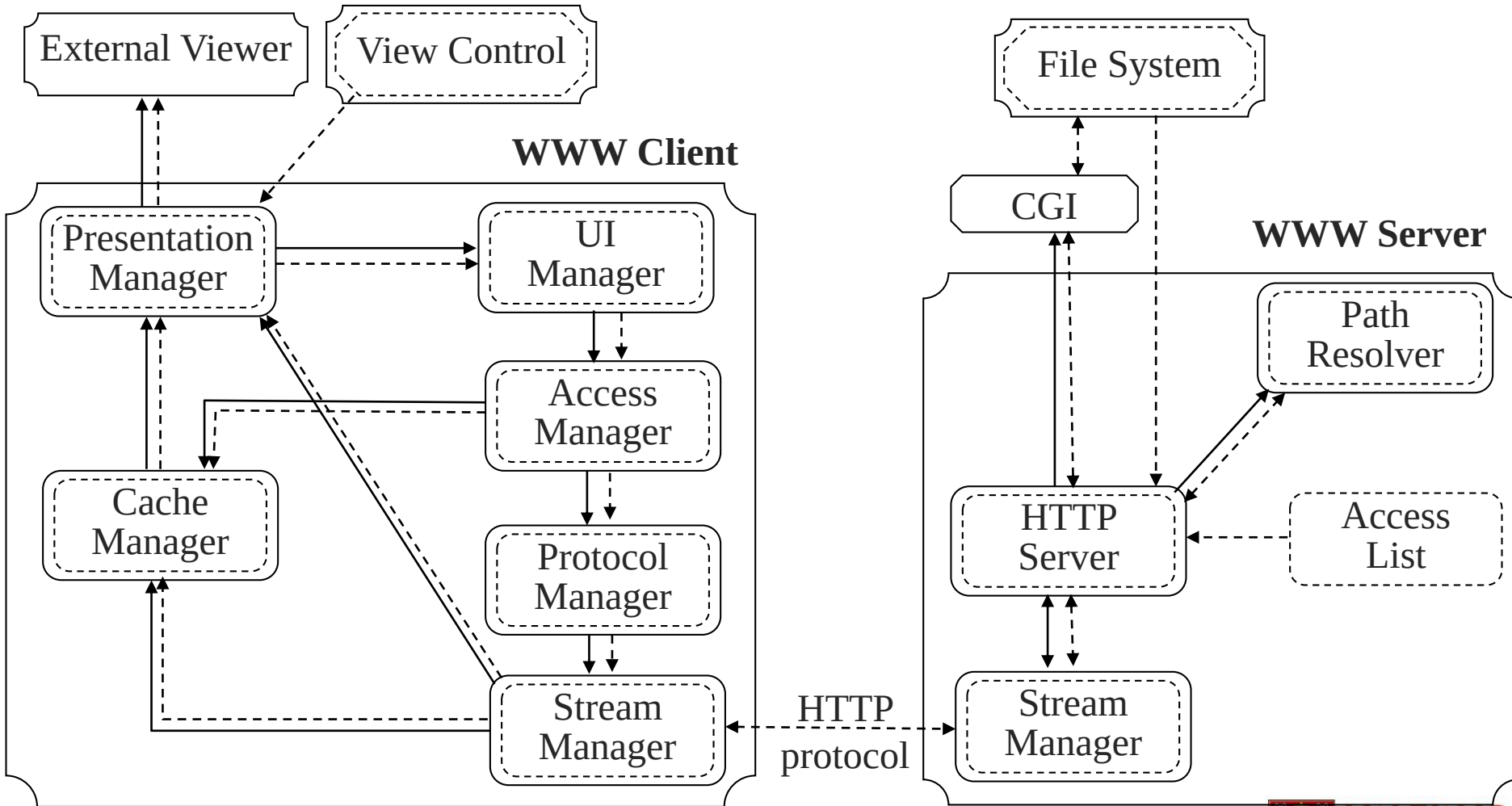
How



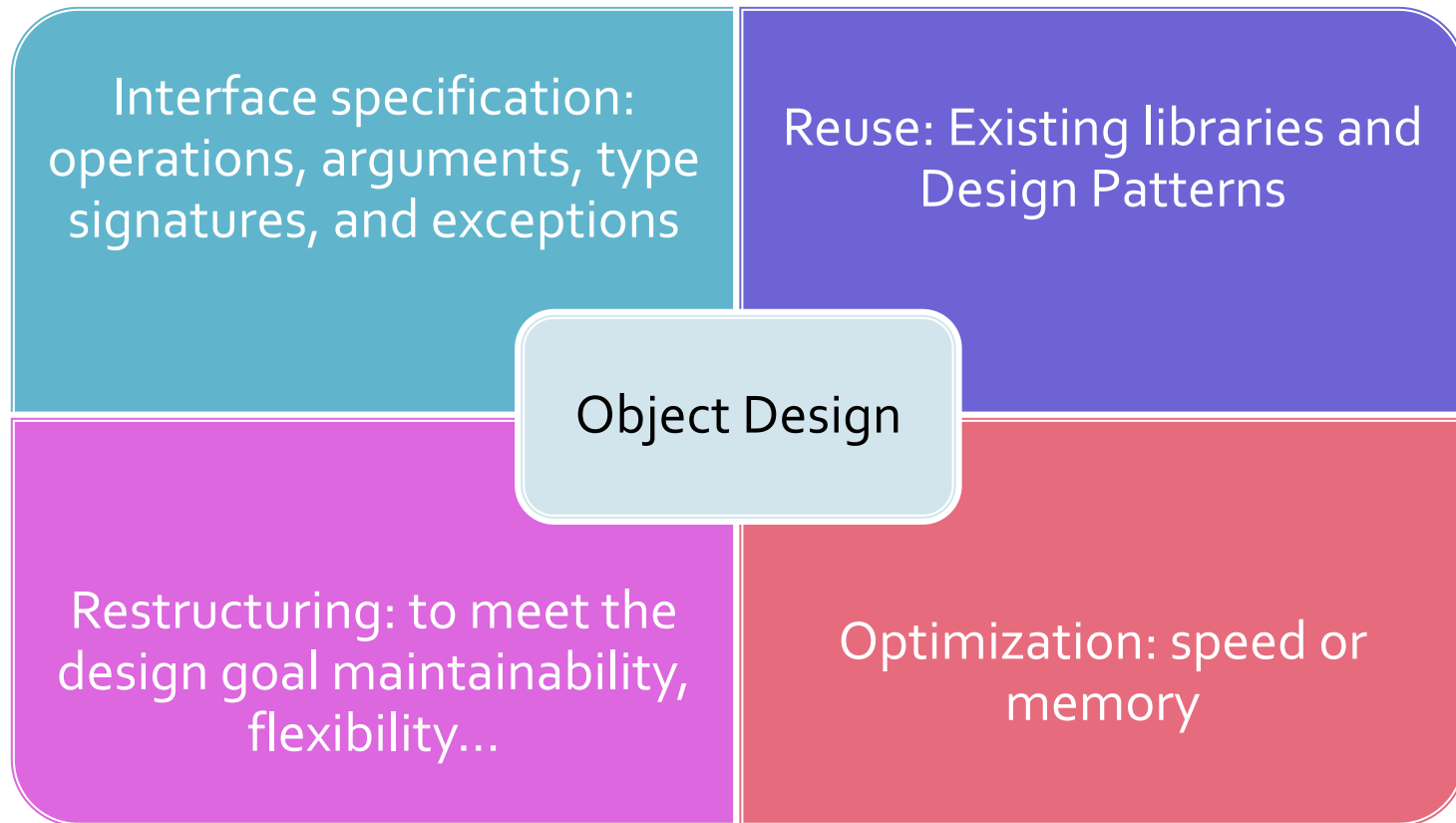
SDLC Activities

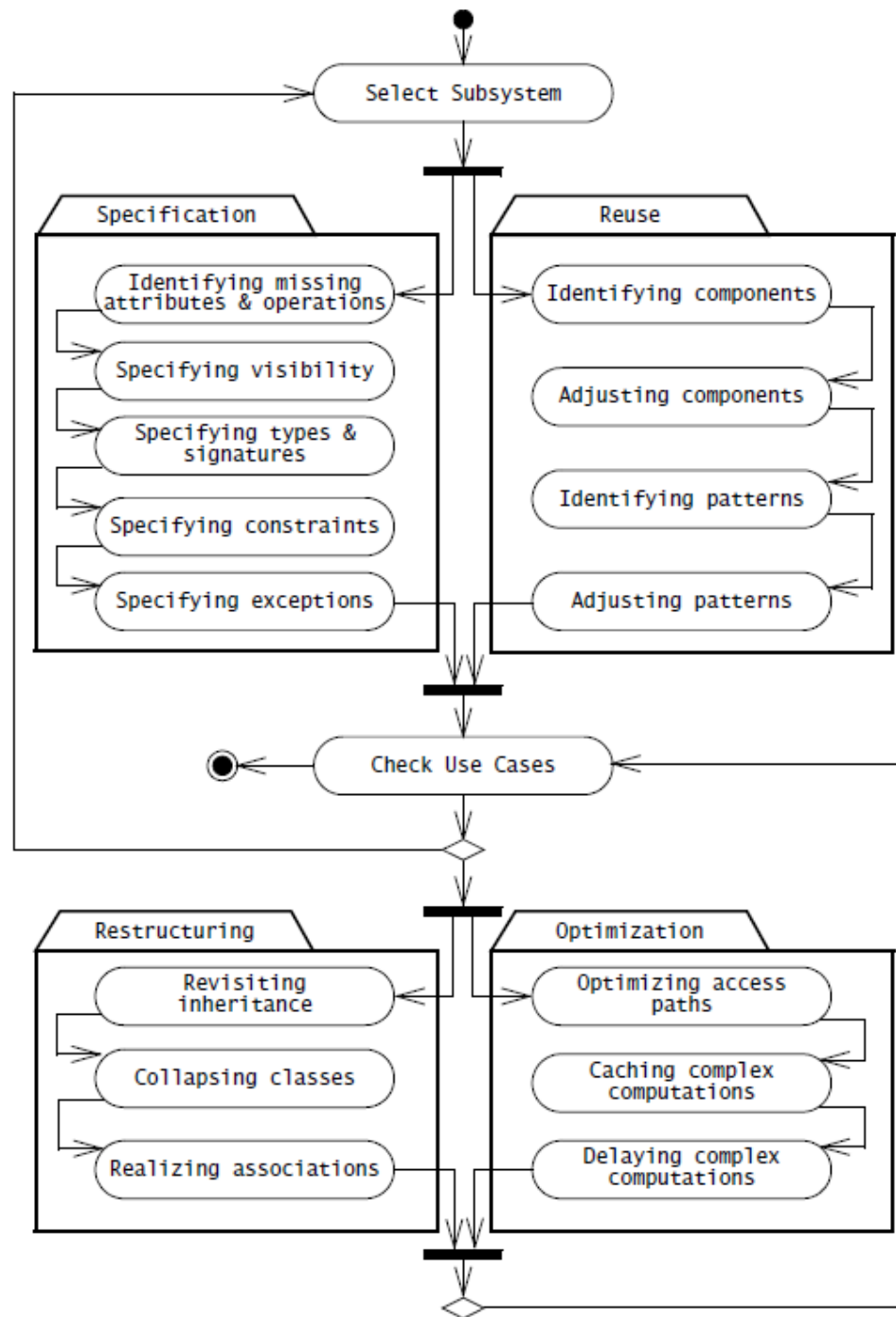


WWW Client-Server Architecture

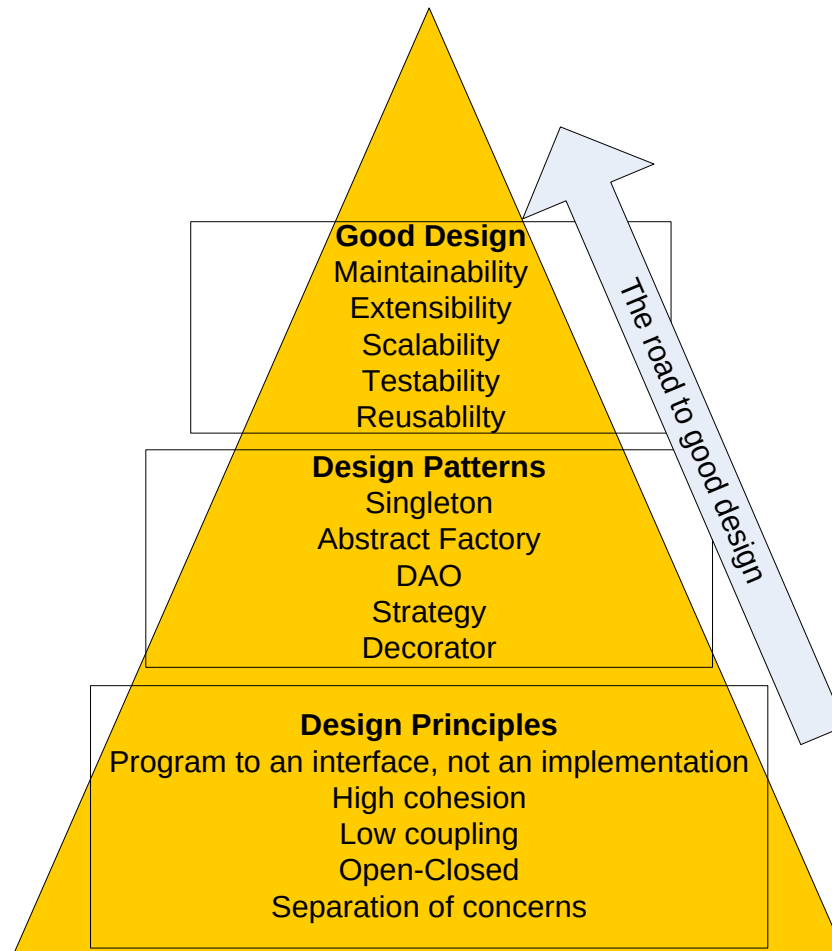


Object Design





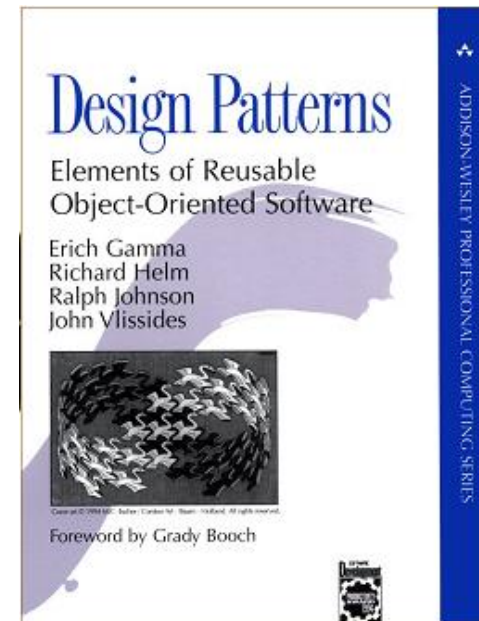
Why use Design Patterns?



What is a Design Pattern?

- A design pattern
 - a proven solution to a problem in a context.
 - abstracts a recurring design structure
 - A template with class and/or object
 - dependencies,
 - structures,
 - interactions, or
 - conventions

“Gang of Four” (GoF) Book, 1996



4 Elements of a Design Pattern

- **Name**
 - Describes the pattern
 - Adds to common terminology for facilitating communication (i.e. not just sentence enhancers)
- **Problem**
 - Describes when to apply the pattern
 - Answers - What is the pattern trying to solve?
- **Solution**
 - Describes elements, relationships, responsibilities, and collaborations which make up the design
- **Consequences**
 - Results of applying the pattern
 - Benefits and Costs
 - Subjective depending on concrete scenarios

GoF Patterns (23)

- *Creational Patterns*

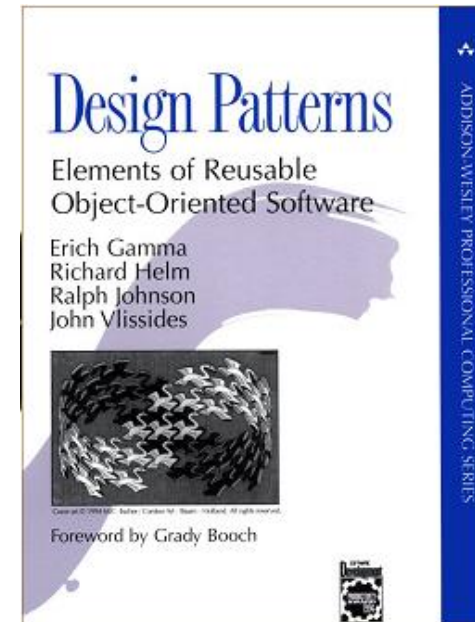
- **Abstract Factory**
- Builder
- Factory Method
- Prototype
- Singleton

- *Structural Patterns*

- Adapter
- Bridge
- Composite
- Decorator
- **Façade**
- Flyweight
- Proxy

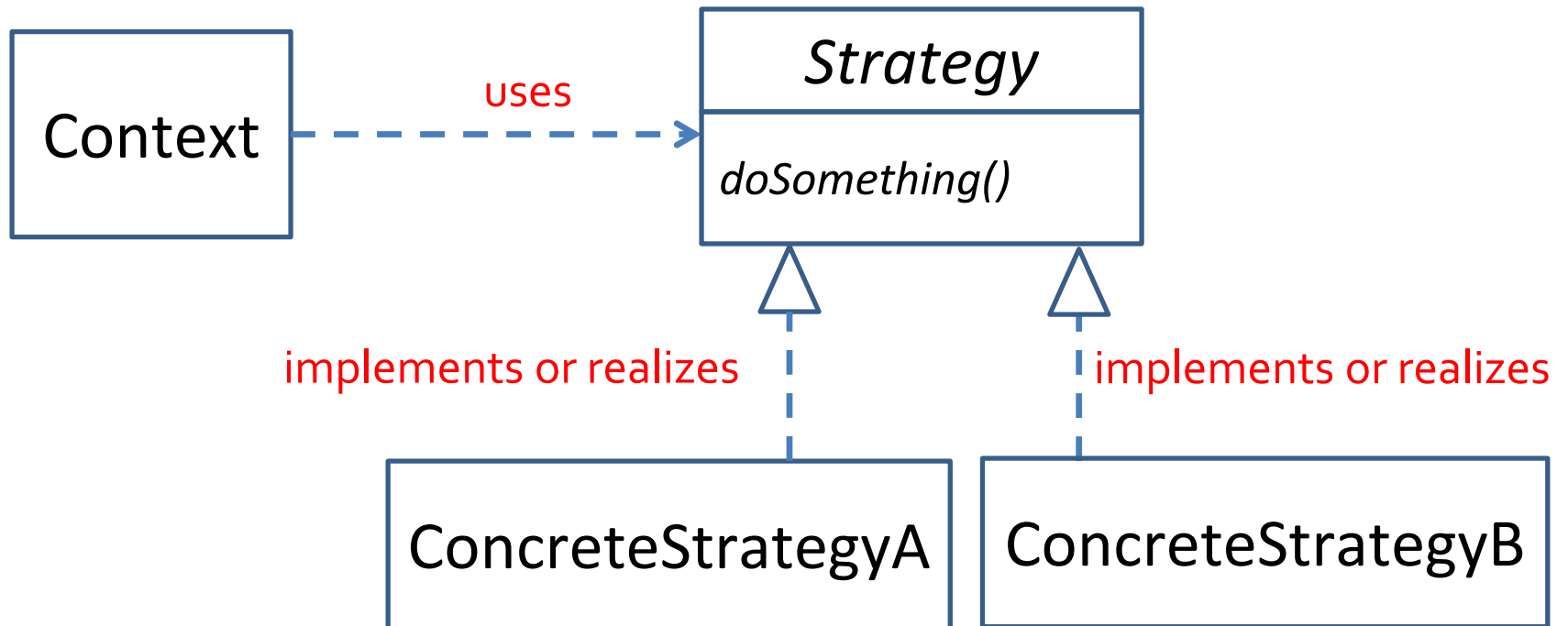
- *Behavioral Patterns*

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- **Observer**
- State
- **Strategy**
- Template Method
- Visitor



Strategy Pattern

- ▶ Design problem: A set of algorithms or objects should be **interchangeable**.
- ▶ Solution: **Strategy Pattern**



Strategy

- Pros
 - Provides encapsulation
 - Hides implementation
 - Allows behavior change at runtime
- Cons
 - Results in complex, hard to understand code if overused

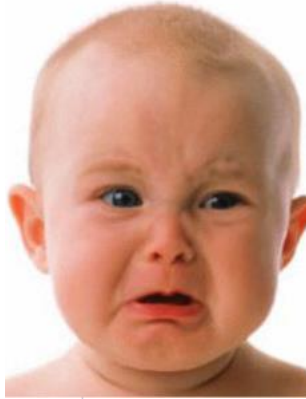
When to Use the Observer Pattern

- ▶ When you need many objects (called **observers**) to receive an **update** when another object (called **subject**) **changes**



a.k.a. **Subscriber** – **Publisher** Pattern
Consumer – **Producer** Pattern

Why to use the Observer Pattern?



Subject

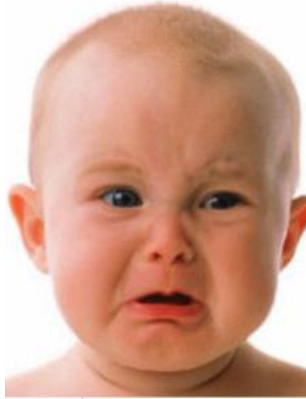
- I do not know (or care) who are looking after me and how many are looking after me
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying



Observer

- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- I cannot constantly check the baby's status because I have to do something else important

Why to use the Observer Pattern?



Subject

- I do not know (or care) who are looking after me and how many are looking after me
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying

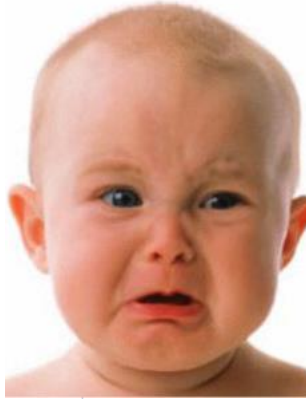
What kind of coupling (tight or loose) is preferred? Why?



Observer

- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- I cannot constantly check the baby's status because I have to do something else important

Why to use the Observer Pattern?



Subject

- *I do not know (or care) who are looking after me and how many are looking after me*
- I want to let whoever are looking after me know when I cry
- It is up to caregivers to decide what to do with me crying

Loose coupling is a benefit for both sides!

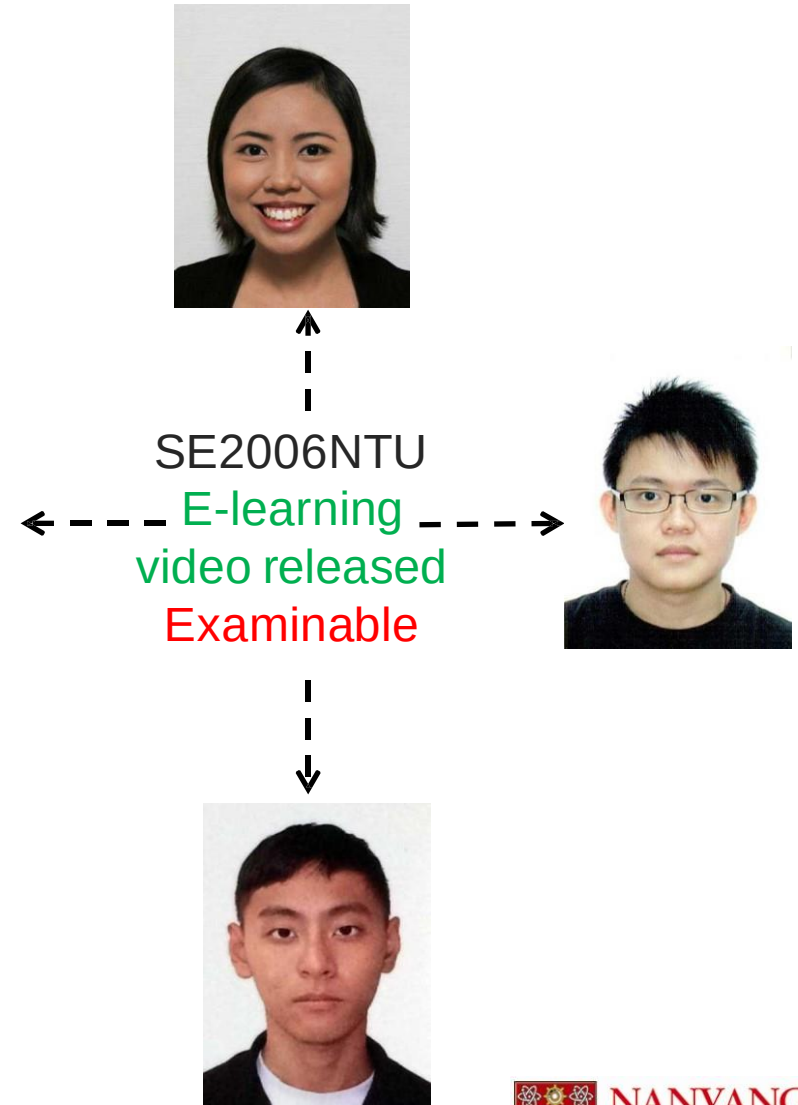


Observer

- Sometimes I want to look after the baby, sometimes I do not
- I want to know the incident once the baby cries, but I do not know when he will cry
- *I cannot constantly check* the baby's status because I have to do something else important

When to Use the Observer Pattern

- ▶ When you need many objects (called **observers**) to receive an **update** when another object (called **subject**) **changes**



- ▶ *Who's the Subject?*
- ▶ *Who's the Observer?*

Why to use the Observer Pattern?



School of Computer Engineering

Subject

- It does not matter who register the course and how many register the course
- I want to let whoever register the course know the latest announcement
- It is up to course takers to decide what to do with the change



Observer

- I want to be able to register/dropoff the course
- I want to know the latest announcement once it occurs, but I do not know when it will occur
- I cannot constantly check the announcement because I have to do something else important

Why to use the Observer Pattern?



School of Computer Engineering

Subject

- It does not matter who register the course and how many register the course
- I want to let whoever register the course know the latest announcement
- It is up to course takers to decide what to do with the change

What kind of coupling (tight or loose) is preferred?



Observer

- I want to be able to register/dropoff the course
- I want to know the latest announcement once it occurs, but I do not know when it will occur
- I cannot constantly check the announcement because I have to do something else important

Why to use the Observer Pattern?



School of Computer Engineering

Subject

- *It does not matter who register the course and how many register the course*
- I want to let whoever register the course know the latest announcement
- It is up to course takers to decide what to do with the change

Loose coupling is a benefit for both sides!

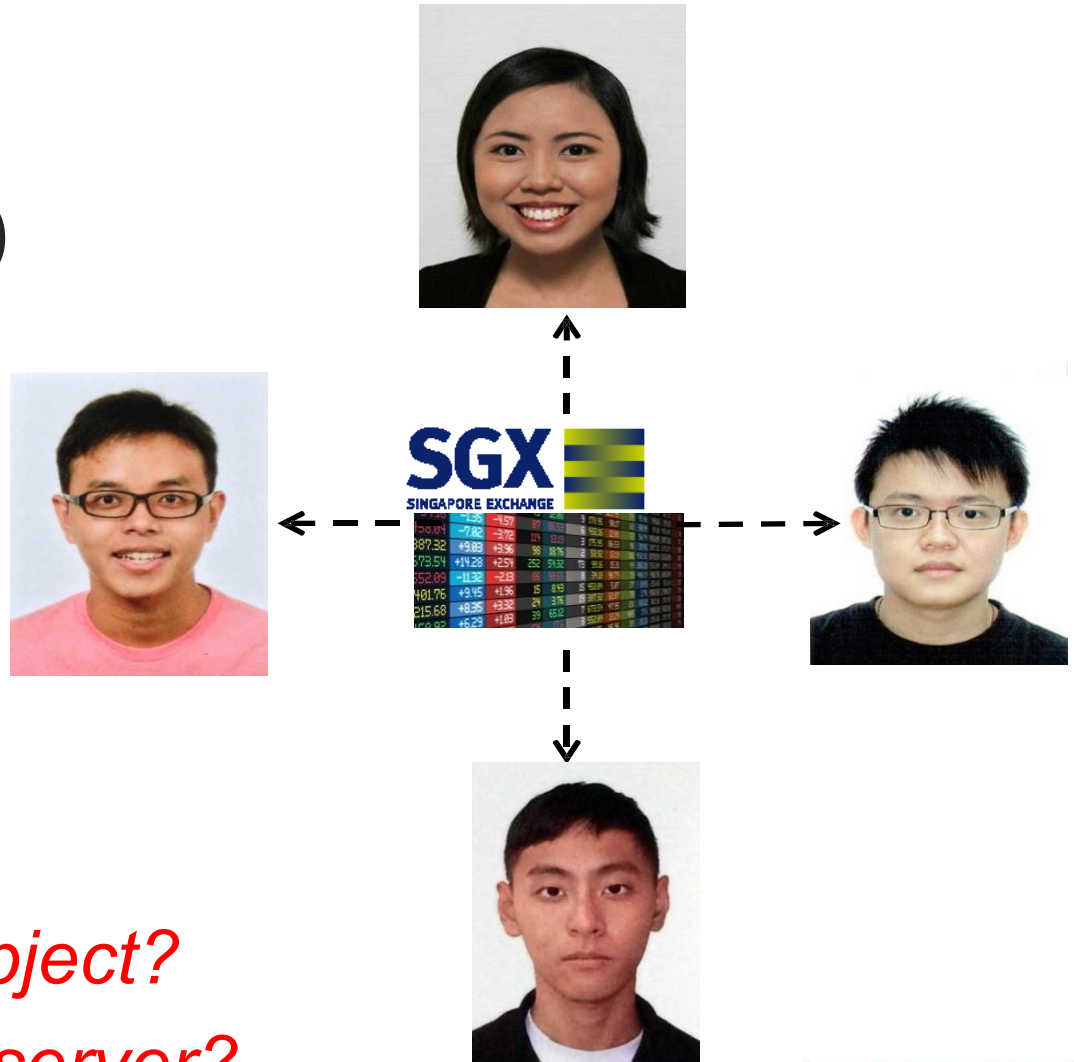


Observer

- I want to be able to register/dropoff the course
- I want to know the latest announcement once it occurs, but I do not know when it will occur
- *I cannot constantly check the announcement* because I have to do something else important

When to Use the Observer Pattern

- ▶ When you need many objects (called **observers**) to receive an **update** when another object (called **subject**) **changes**



- ▶ *Who's the Subject?*
- ▶ *Who's the Observer?*

Why to use the Observer Pattern?



57	87	86.53	9	17
72	114	13.19	6	43
96	98	18.76	3	17
54	252	54.32	2	5
13	86	98.65	73	9
96	15	8.43	8	3
32	24	3.76	15	45
03	39	65.12	19	38
	54	17.6	7	67
			3	55

Subject

- I do not care who invest in the market and how many invest in the market
- I want to let whoever invest in the market know the latest stock change
- It is up to investors to decide what to do with the change



Observer

- I want to be able to invest in or withdraw from the market freely
- I want to know the latest stock change once it occurs, but I do not know when it will occur
- I cannot constantly check the change because I have to do something else important

Why to use the Observer Pattern?



57	87	86.53	9	17
72	114	13.19	6	43
96	98	18.76	3	17
54	252	54.32	2	5
13	86	98.65	73	9
96	15	8.43	8	3
32	24	3.76	15	45
03	39	65.12	19	38
	54	17.6	7	67
			3	55

Subject

- *I do not care who invest in the market and how many invest in the market*
- I want to let whoever invest in the market know the latest stock change
- It is up to investors to decide what to do with the change

Loose coupling is a benefit for both sides!



Observer

- I want to be able to invest in or withdraw from the market freely
- I want to know the latest stock change once it occurs, but I do not know when it will occur
- *I cannot constantly check the change* because I have to do something else important

Observer Pattern – Design Problems

- ▶ Defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

Easy Subscription and Automatic Notification

How to Resolve these Problems?

- ▶ **Subscription** mechanism
 - ▶ Observers freely register/unregister their interests in Subject
- ▶ **Notification** mechanism
 - ▶ Subject propagates the change to Observer when the change occurs

Understand How it Works: NTULearn Case Study

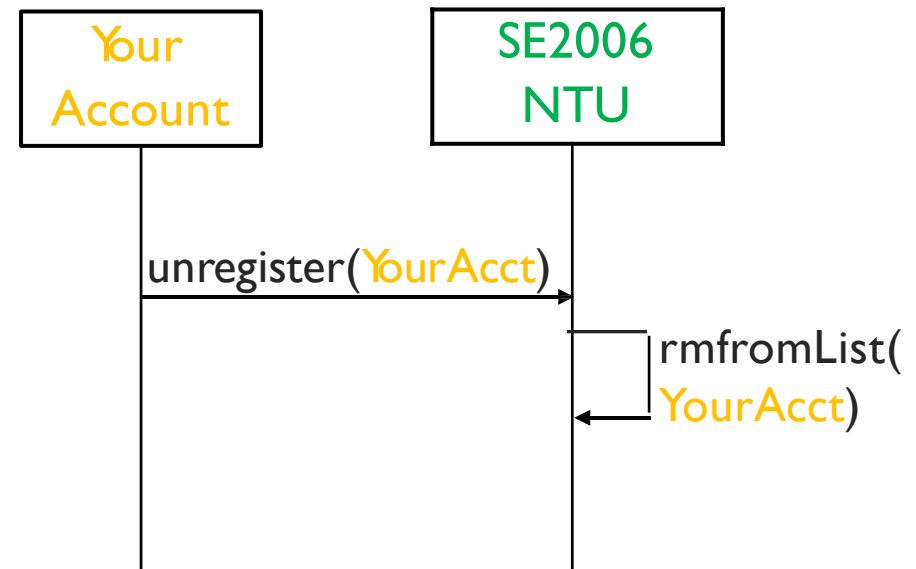
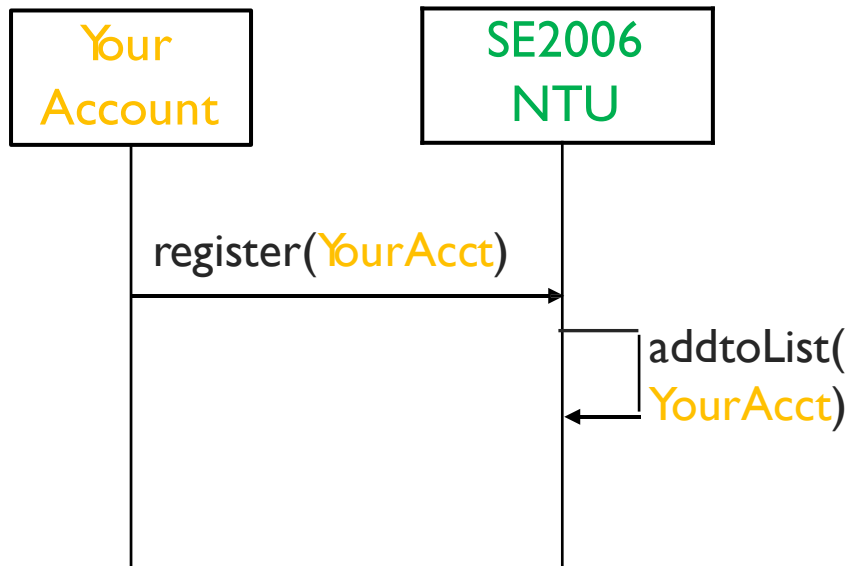
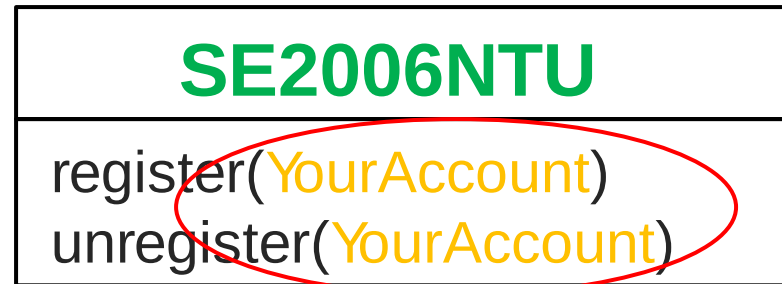
- ▶ In NTULearn system
 - ▶ **Subject:** SE2006NTU
 - ▶ **Observers:** Course takers (your NTU account)

SE2006NTU

YourAccount

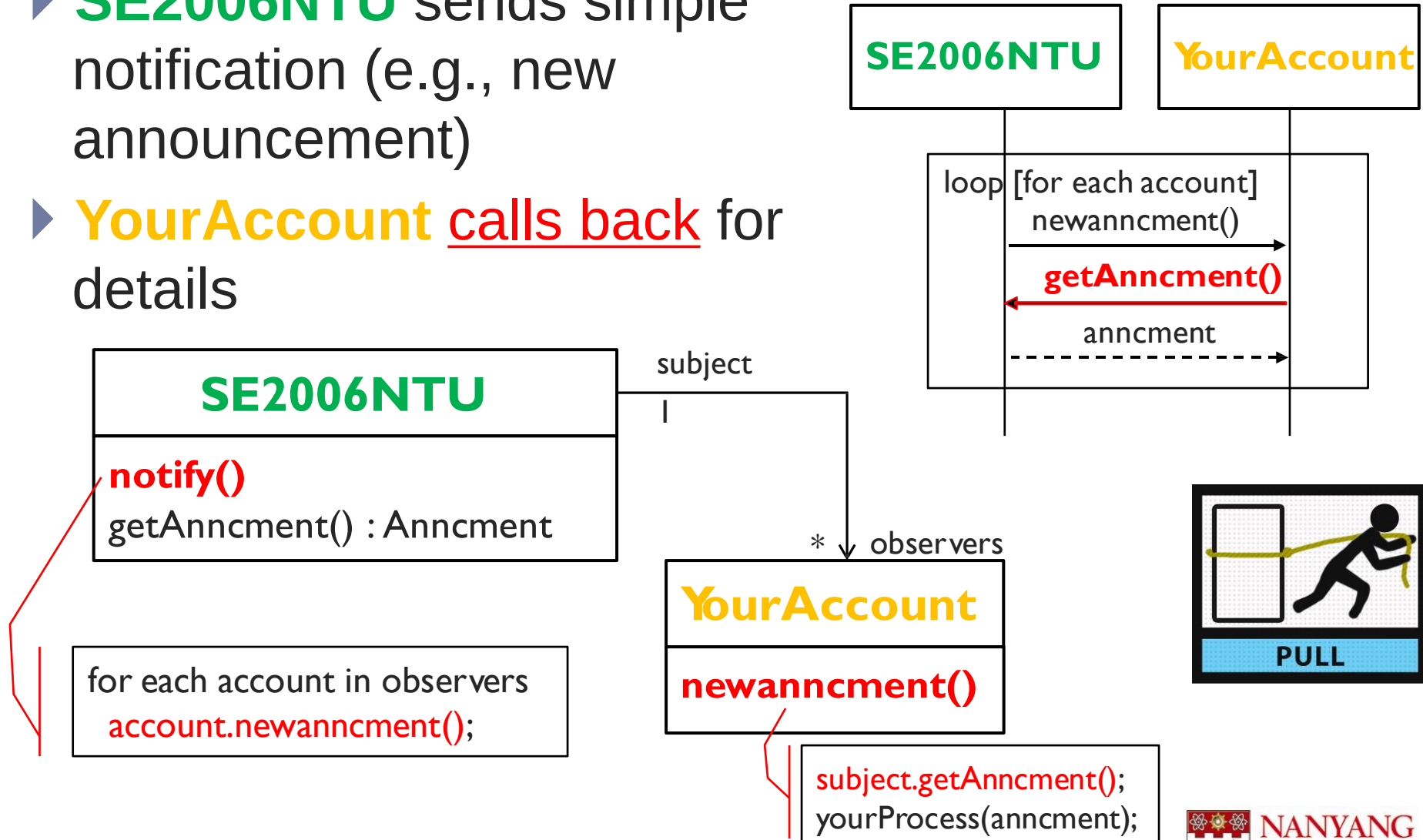
Subscription Mechanism

- ▶ **Add to course**
(i.e., register)
- ▶ **Remove from course**
(i.e., unregister)



Notification Mechanism – *Pull Update*

- ▶ **SE2006NTU** sends simple notification (e.g., new announcement)
- ▶ **YourAccount** calls back for details



Pull it Together in a Class Diagram

SE2006NTU

register(YourAccount)
unregister(YourAccount)
notify()
getAnncmnt() : Anncmnt

for each account in observers
account.newanncmnt();

subject

observers

1

*

YourAccount

newanncmnt()

subject.getAnncmnt();
yourProcess(anncmnt);

SE3003NTU

register(YourAccount)
unregister(YourAccount)
notify()
getAnncmnt() : Anncmnt

for each account in observers
account.newanncmnt();

subject

observers

1

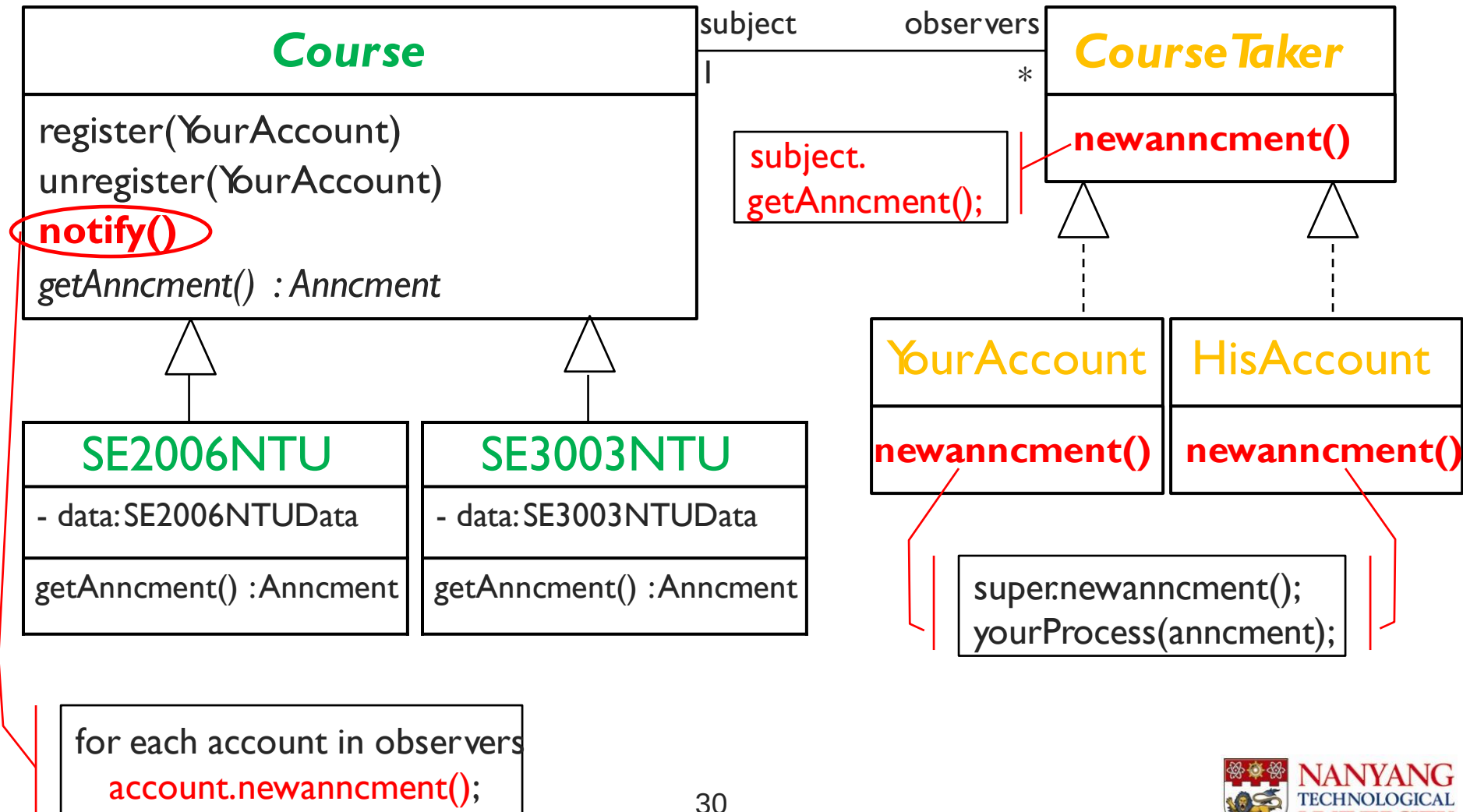
*

HisAccount

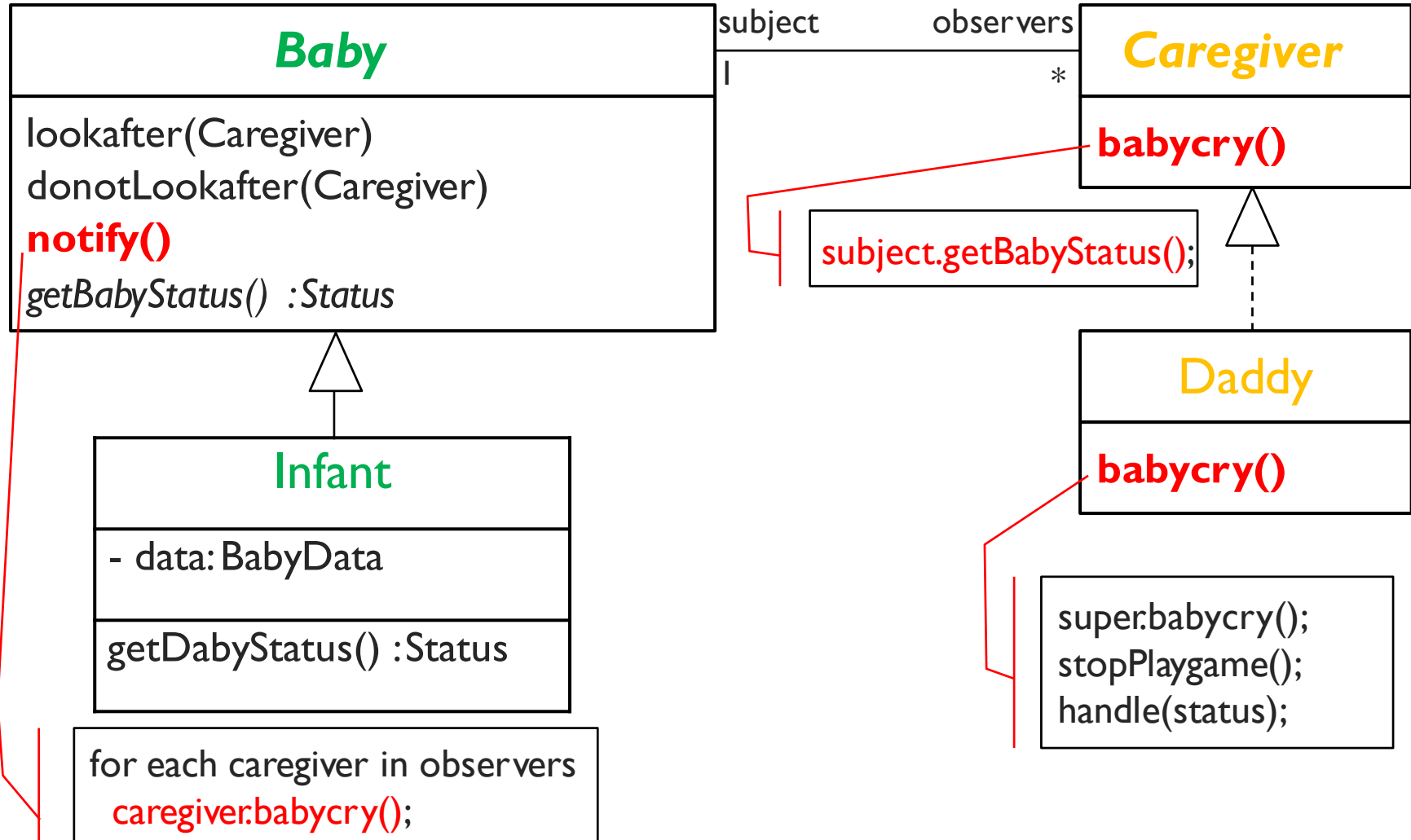
newanncmnt()

subject.getAnncmemnt();
hisProcess(anncmnt);

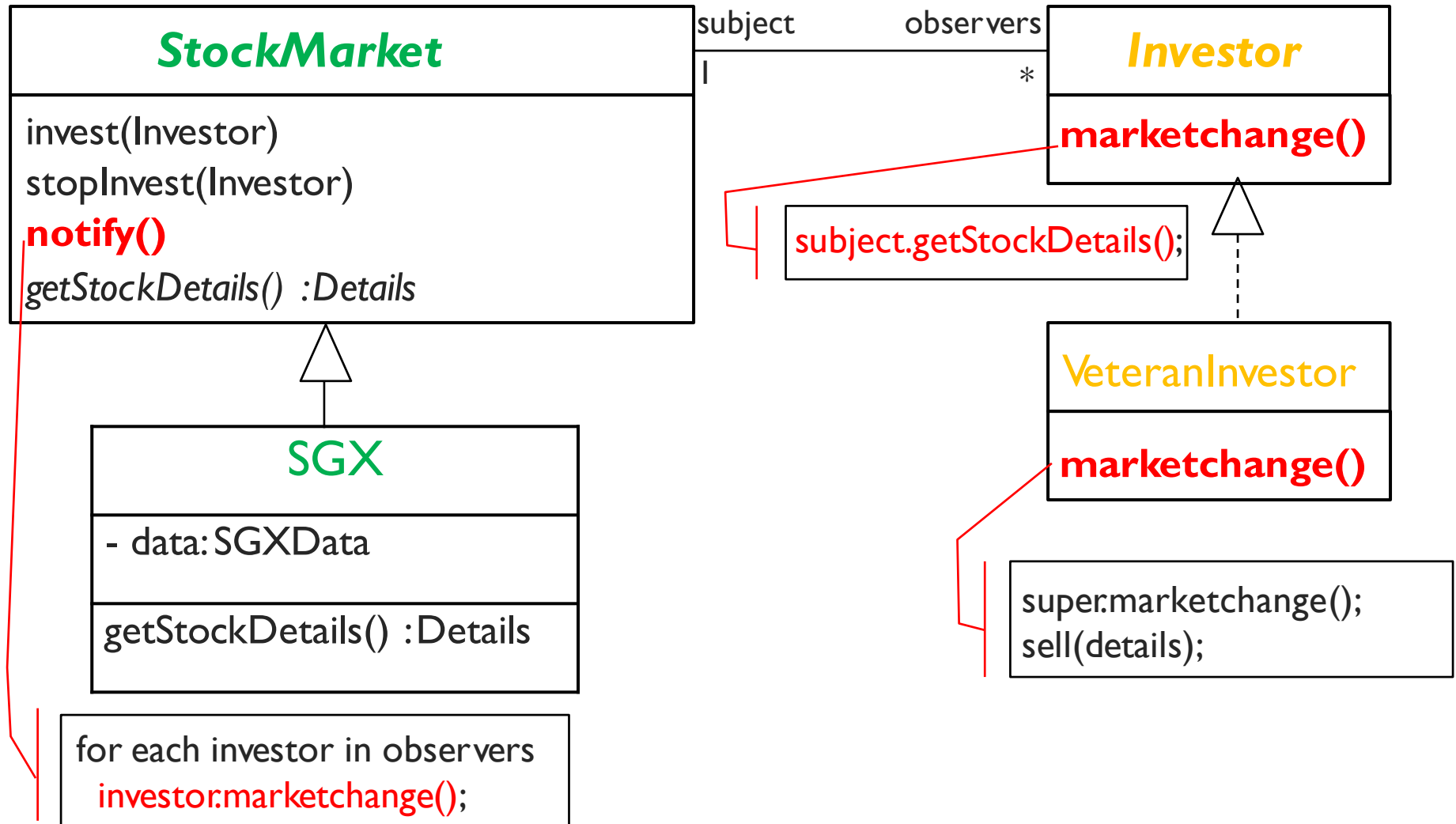
Abstract Commonalities into Superclasses – CourseTaker – Course Design



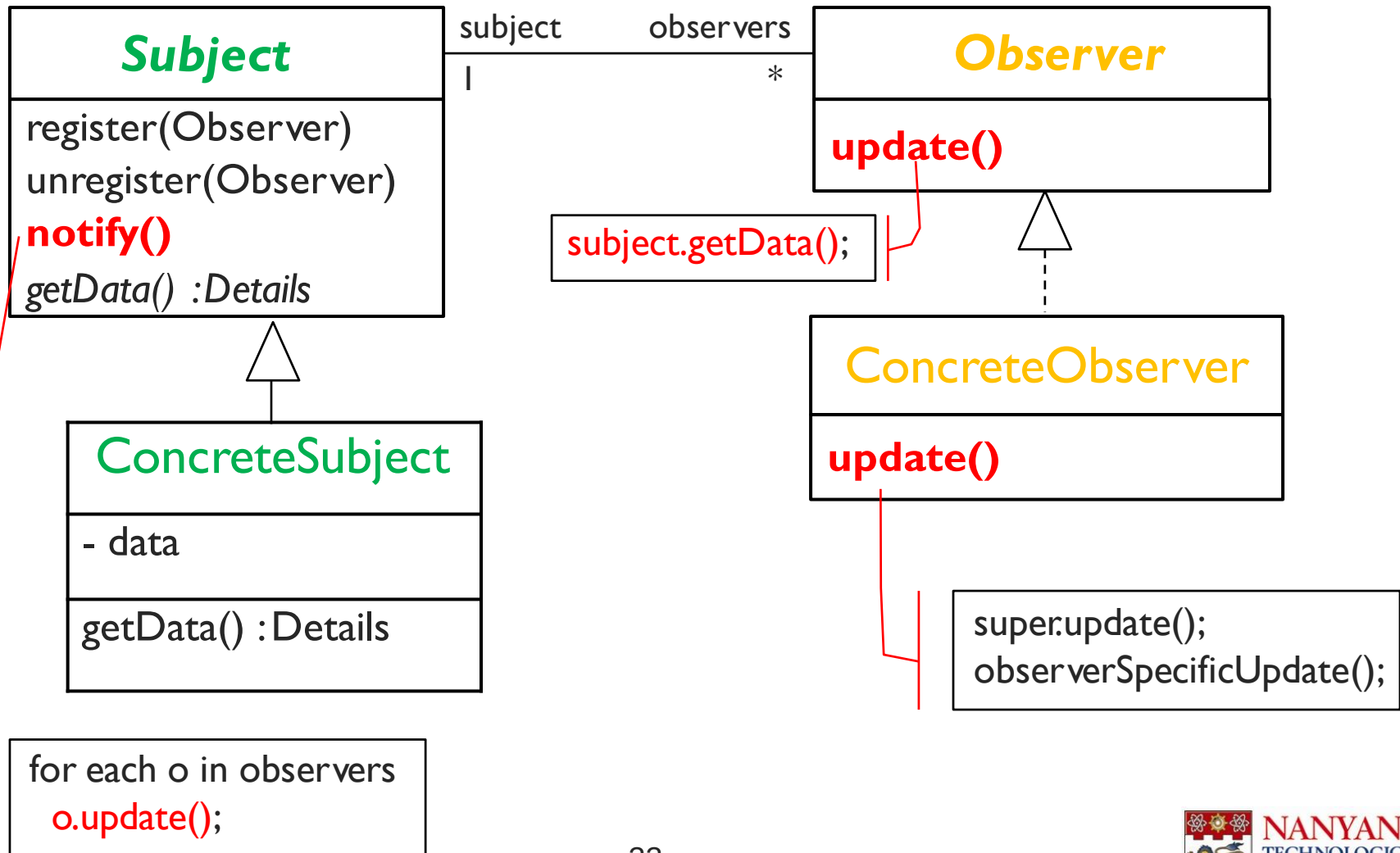
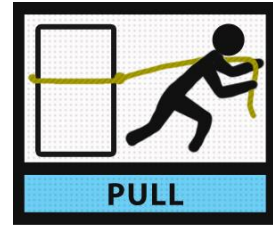
Caregiver – Baby Design



Investor – StockMarket Design

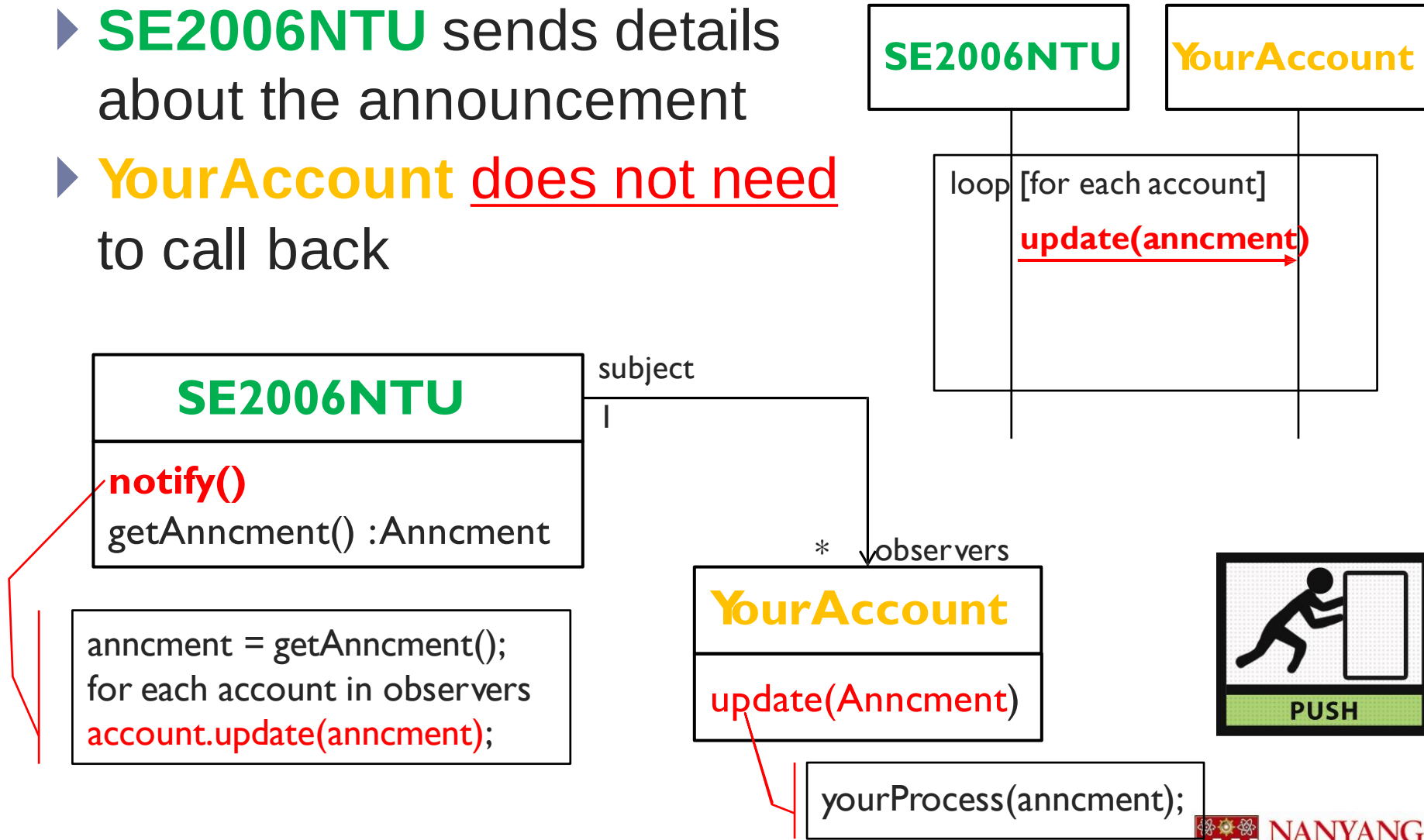


Observer Pattern Template

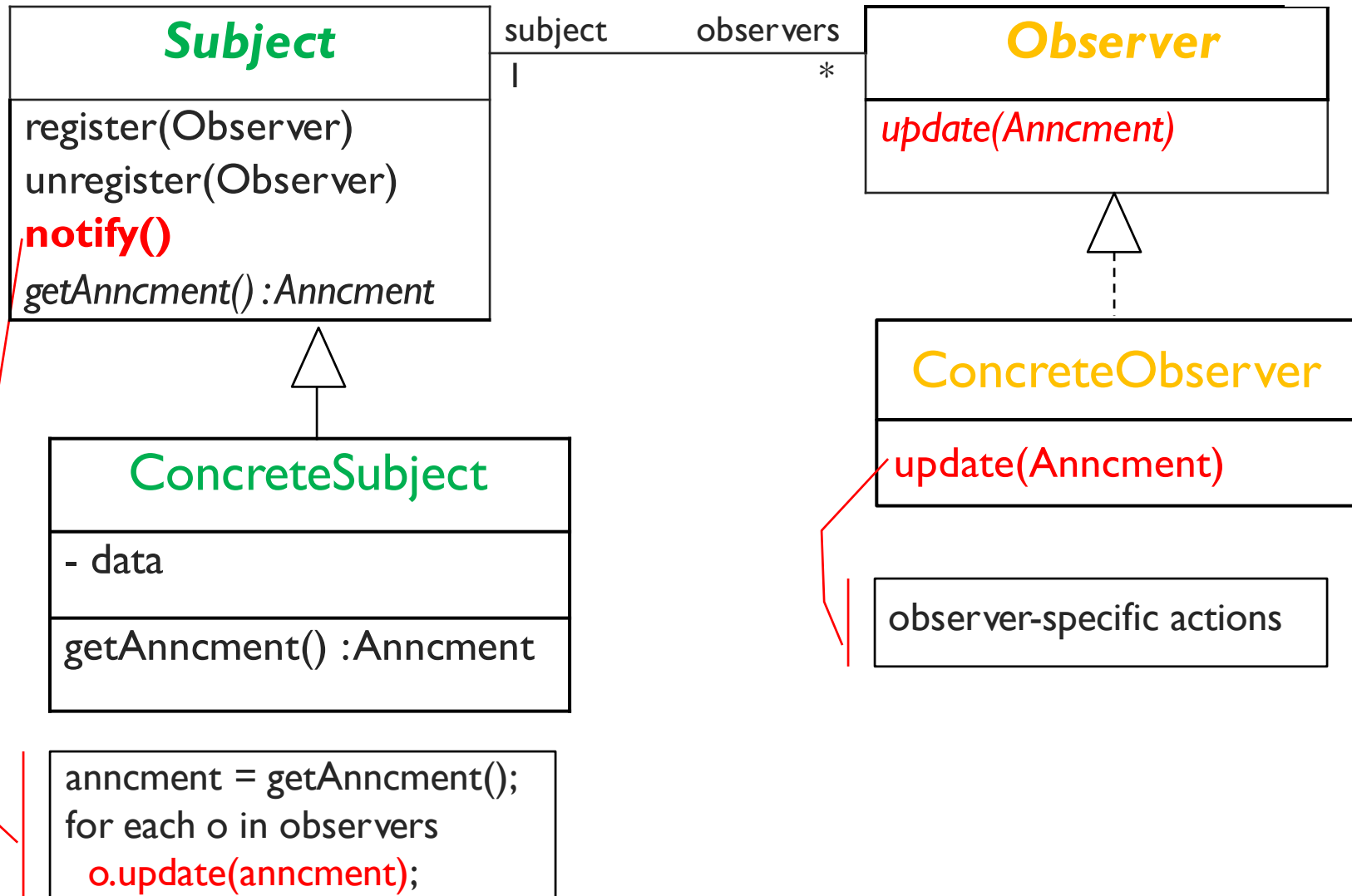
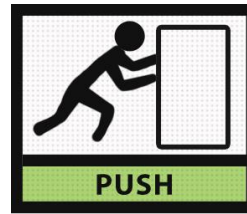


Notification Mechanism – *Push Update*

- ▶ **SE2006NTU** sends details about the announcement
- ▶ **YourAccount** does not need to call back



Observer Pattern Template



Update Protocol: Pull or Push?

Which one is better?

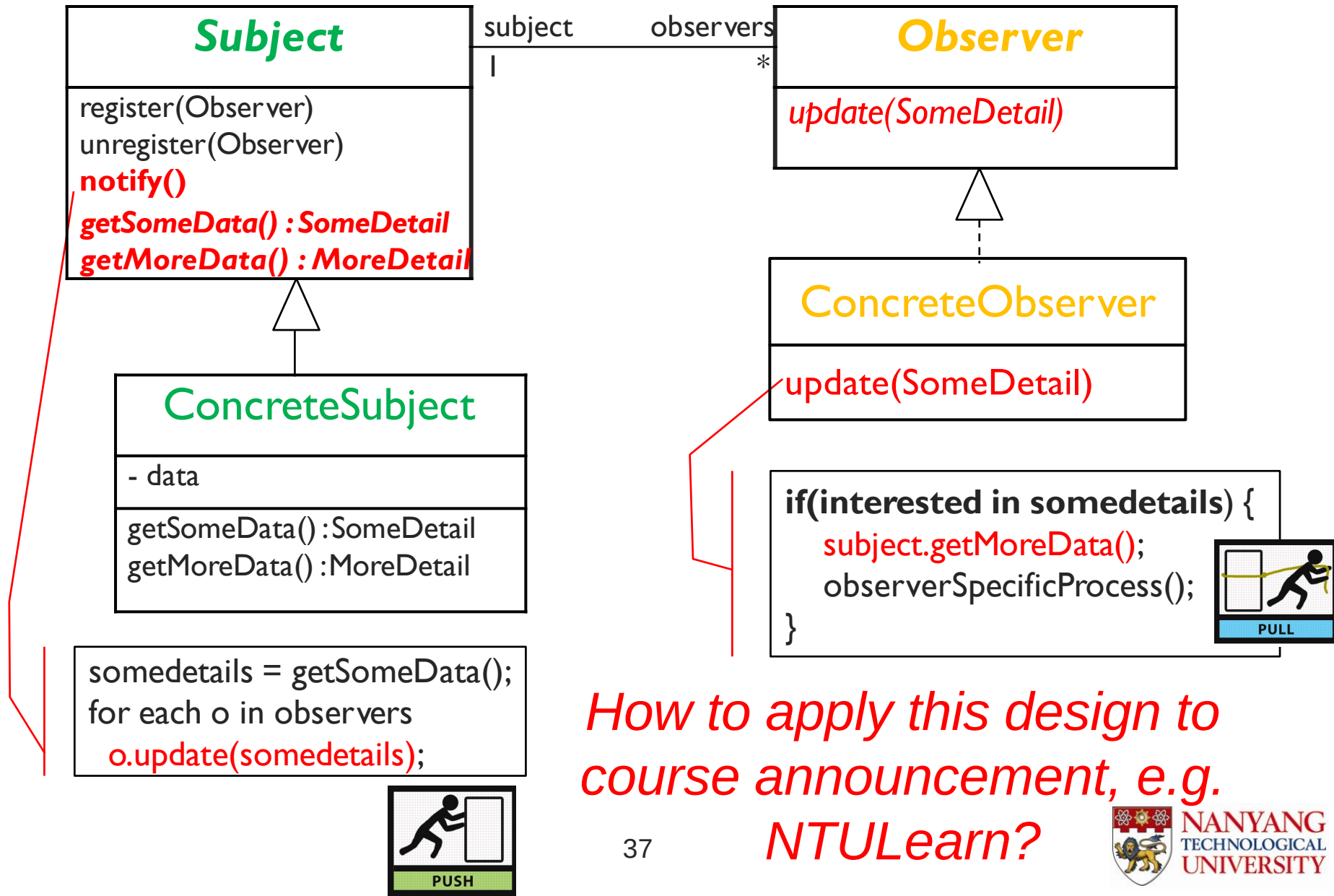
► Pull update

- Subject sends simple notification (via `Observer.update()`).
- The Observer **calls back** (via `Subject.getData()`) for details explicitly (if Observer is interested) – *two way communication*.

► Push update

- Subject sends detailed information about the change (or update), whether Observer wants it (interested) or not – Observer **does not need** to call back – *one way communication*.

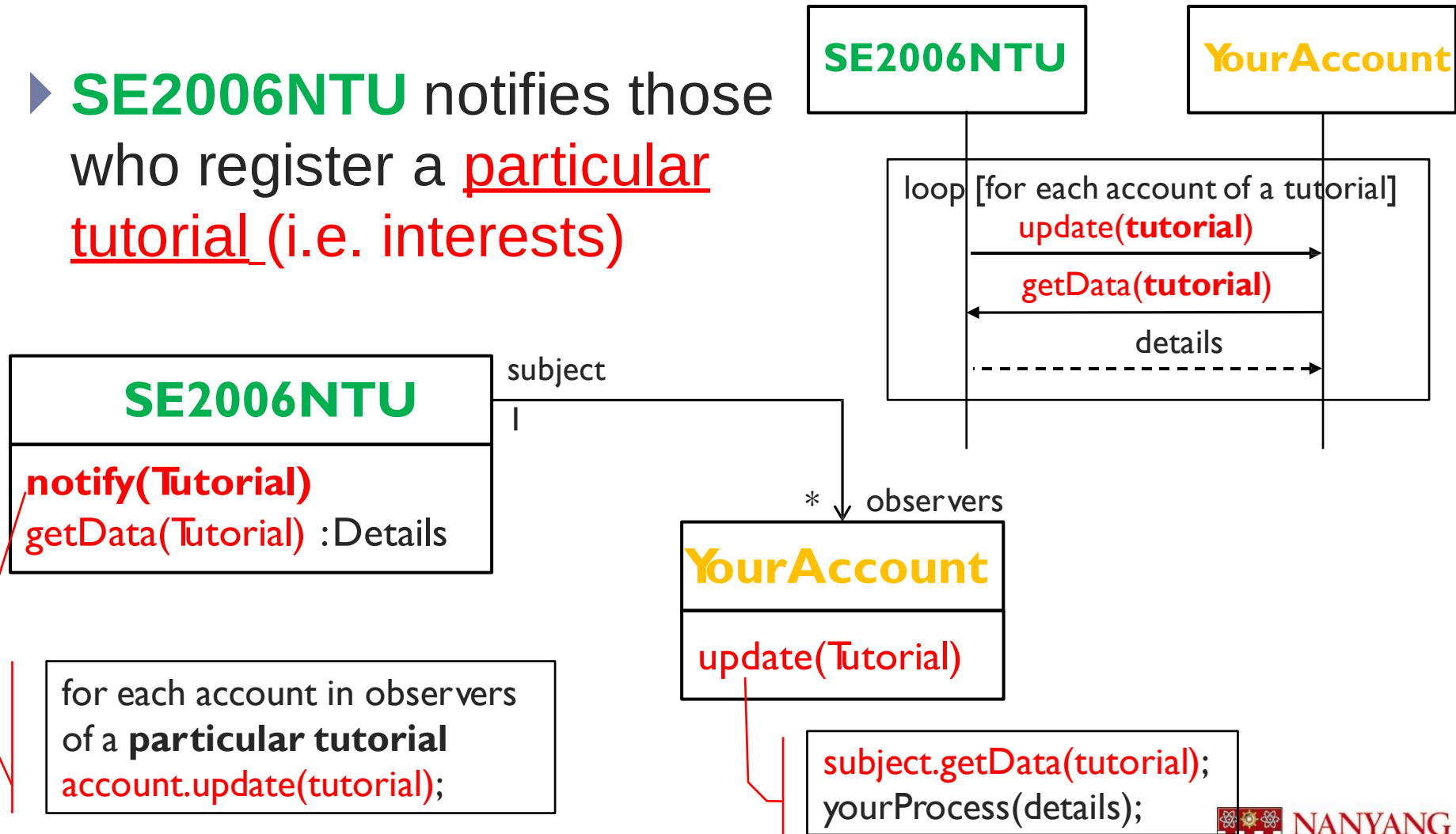
Observer Pattern – *Push + Pull Update*



*How to apply this design to
course announcement, e.g.
NTULearn?*

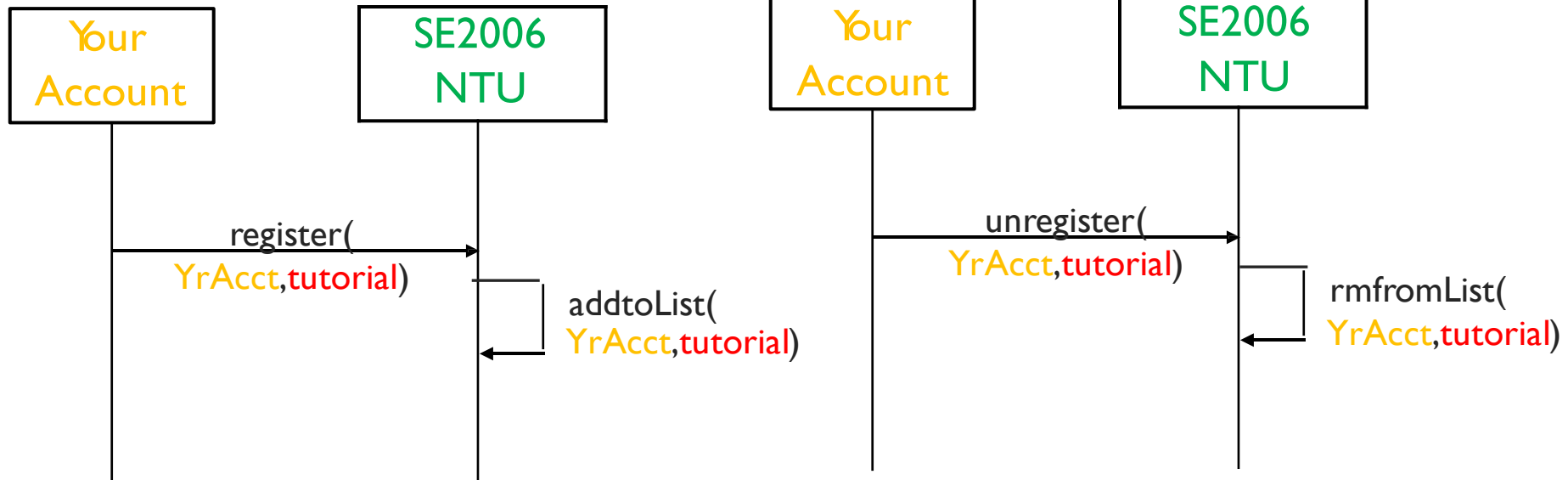
Notification Mechanism – *Selective Notification*

- ▶ **SE2006NTU** notifies those who register a particular tutorial (i.e. interests)



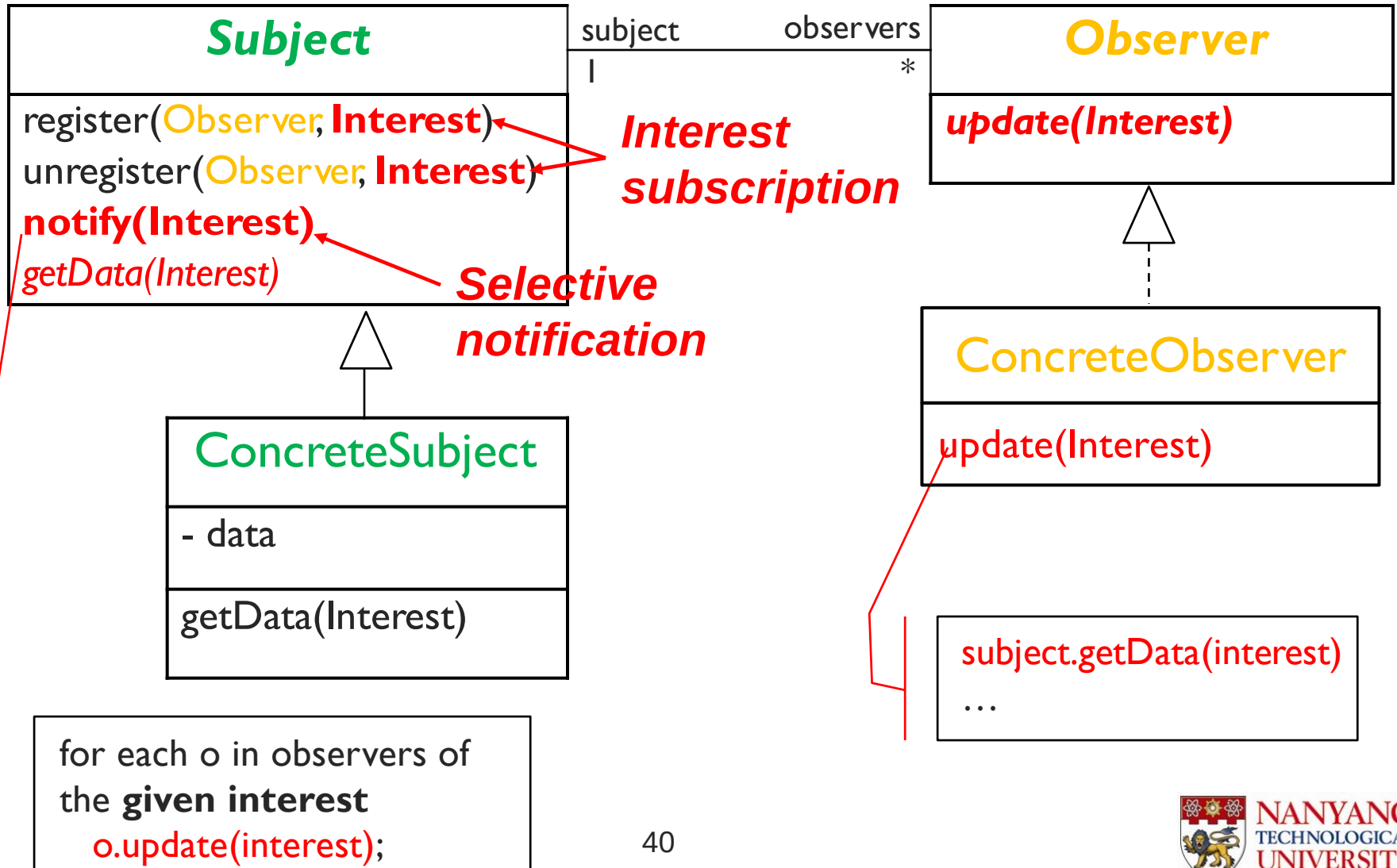
Subscription Mechanism – *Interests Subscription*

- ▶ Add to course
(i.e., register)
- ▶ Remove from course
(i.e., unregister)



Indicate your **tutorial** session when registering the course

Observer Pattern – *Interest Subscription + Selective Notification*



Event Handling – Design Problems

- ▶ **Loose coupling** between an **event source** (**subject**) and its dependent **event handlers** (**observers**)
- ▶ **Event source** wants to notify **event handlers** the event once it occurs, but event handlers can change their interests in event source freely
- ▶ **Event handlers** want to know the event once it occurs, but they do not know when the event will occur in event source

Event Handling & Observer Pattern – Design Problems

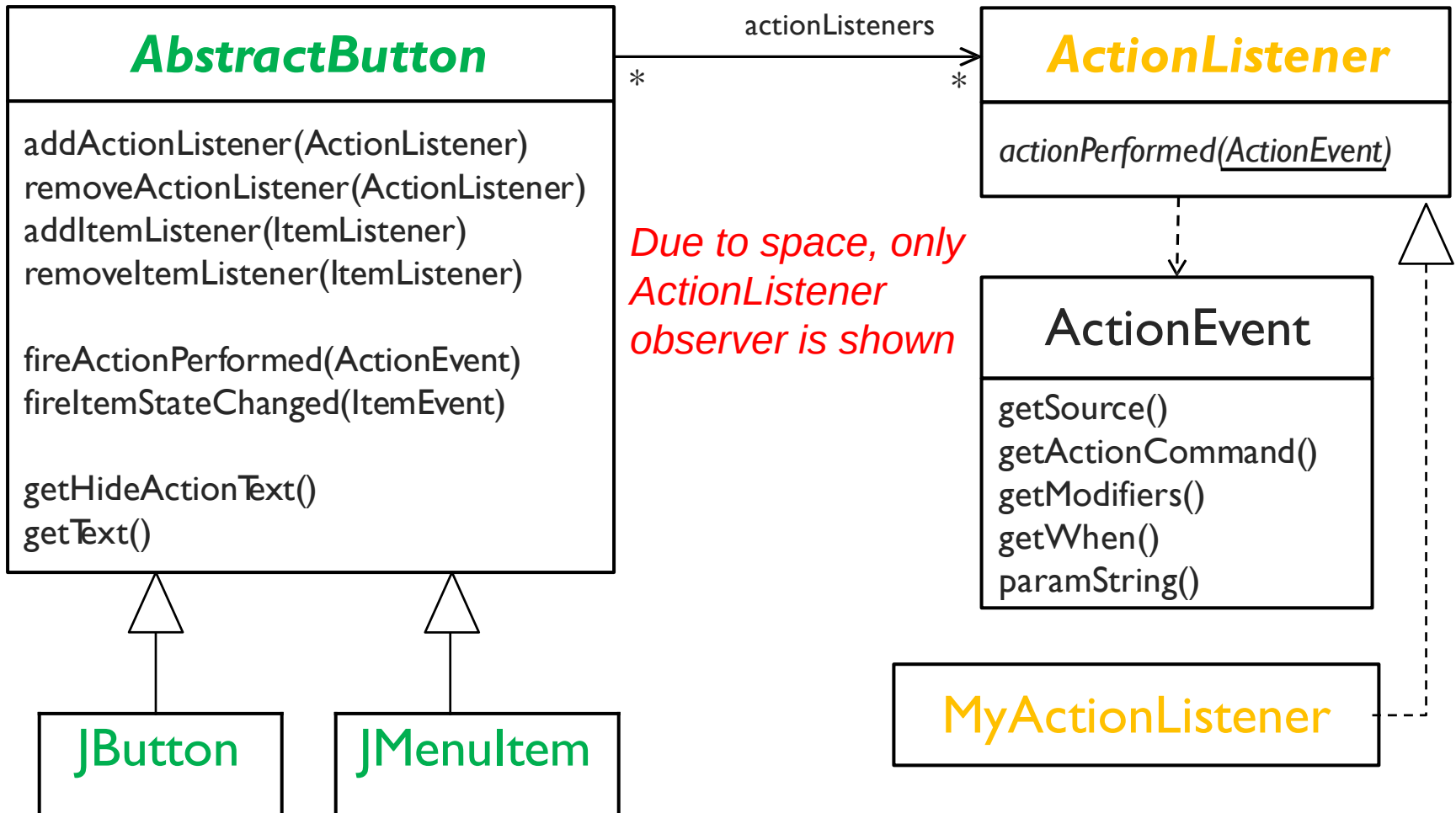
Event Handling

- ▶ **Loose coupling** between an **event source** (**subject**) and its dependent **event handlers** (**observers**)
- ▶ **Event source** wants to notify **event handlers** the event once it occurs, but event handlers can change their interests in event source freely
- ▶ **Event handlers** want to know the event once it occurs, but they do not know when the event will occur in event source

Observer Pattern

- ▶ **Loose coupling** between an **object** (**subject**) and its dependent **objects** (**observers**)
- ▶ **Subject** wants to notify **observers** the change once it occurs, but observers can change their interests freely and constantly
- ▶ **Observers** want to know the change once it occurs, but they do not know when the change will occur

Java Swing Event Handling

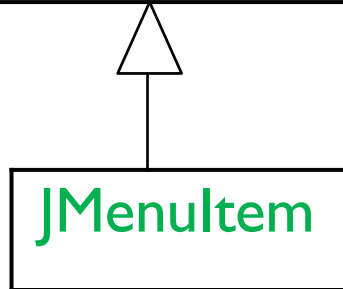
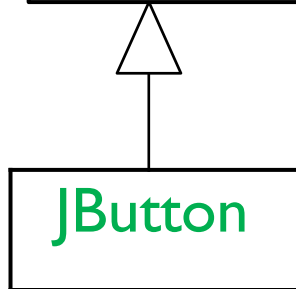
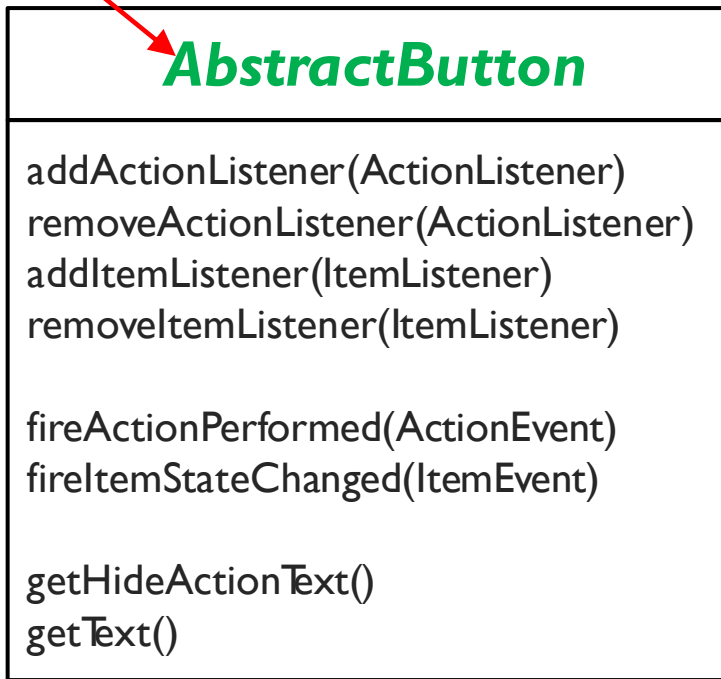


What are the **event sources**?

What are the **event handlers**?

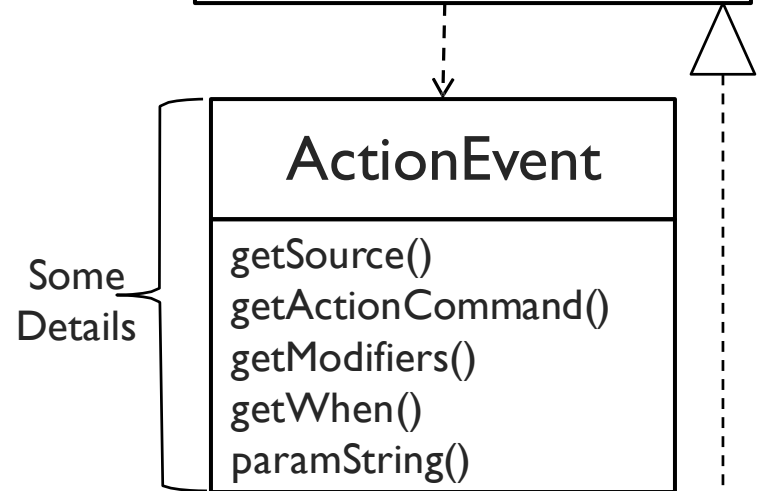
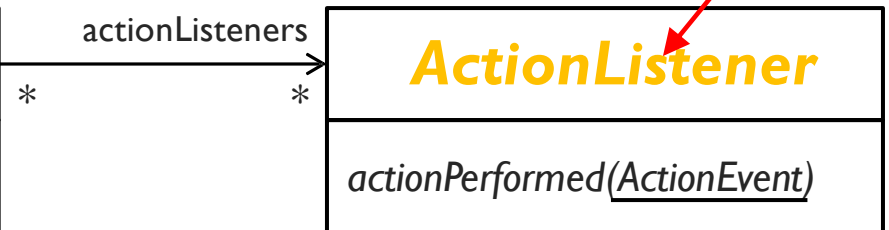
Java Swing Event Handling

Subject



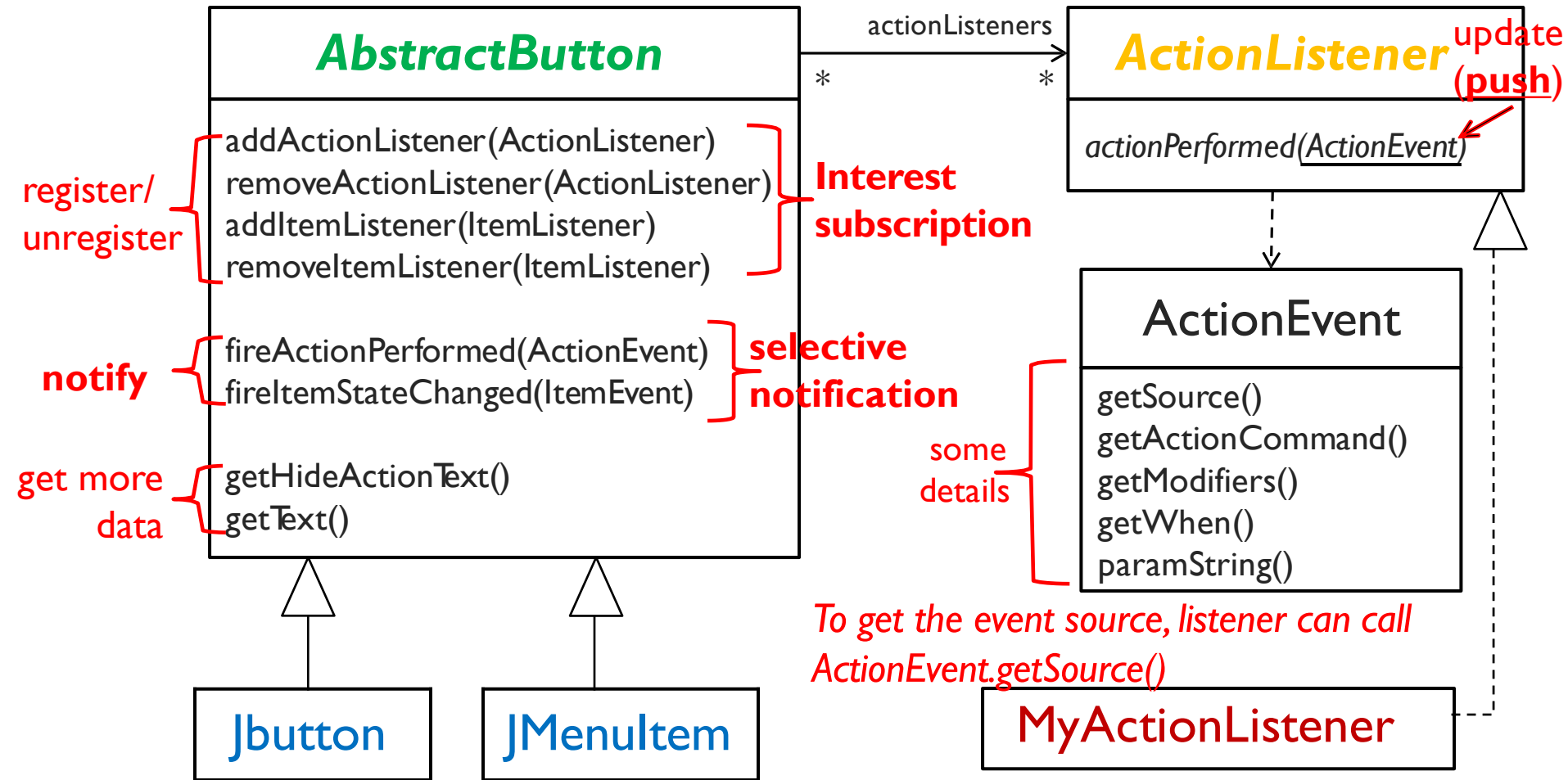
Concrete subjects 44

Observer



Concrete observer

Java Swing Event Handling



- Register different interests
- Selective notification
- Push + Pull update

To get more information, listener can pull more information, e.g. `ActionEvent.getSource().getText()`

Observer

■ Pros

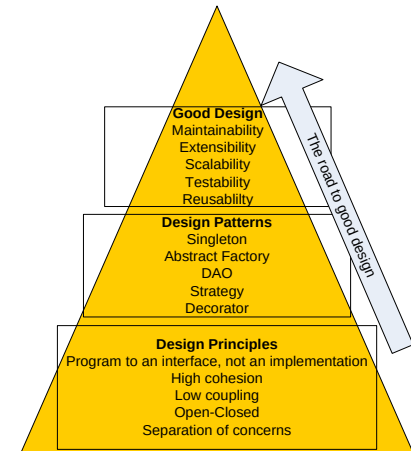
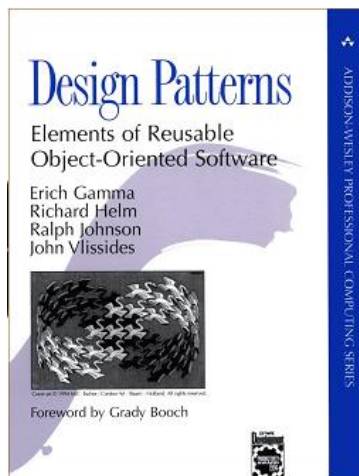
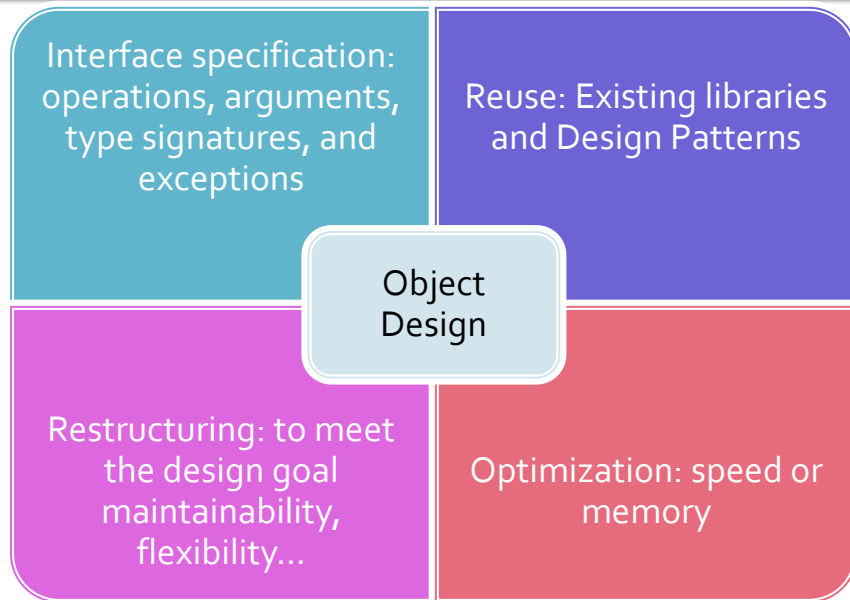
- Abstracts coupling between Subject and Observer
- Supports broadcast communication
- Enables reusability of subjects and observers independently of each other

Loose coupling is a benefit for both sides!

■ Cons

- Slower performance
- If not used carefully the observer pattern can add unnecessary complexity

Summary of Object Design



4 Elements of a Design Pattern

- **Name**
 - Describes the pattern
 - Adds to common terminology for facilitating communication (i.e. not just sentence enhancers)
- **Problem**
 - Describes when to apply the pattern
 - Answers - What is the pattern trying to solve?
- **Solution**
 - Describes elements, relationships, responsibilities, and collaborations which make up the design
- **Consequences**
 - Results of applying the pattern
 - Benefits and Costs
 - Subjective depending on concrete scenarios